

AI 大模型应用开发学习路线 | 26 年最新零基础到精通一条龙（万人收藏★）

编程导航学习网站: [学编程、做项目、拿 Offer!](#)

企业高频面试题库: [开始刷题, 面试遇原题!](#)

精选简历模板大全: [1 分钟搞定简历!](#)

AI 资源导航网站: [获取最新 AI 黑科技!](#)

1 对 1 模拟面试: [随时随地提升面试能力](#)

AI 大模型应用开发求职高频面试题: [开始刷题](#)

★ 推荐先观看 [鱼皮的保姆级 AI 指南视频](#), 快速掌握程序员必知必会的 AI 概念、AI 工具、AI 开发技术、AI 编程技巧。

校招&社招

后端转 AI 应用求职训练营

系统学习 + AI 项目实战 + 求职包装

你可以获得:

- ✓ 定制后端转 AI 学习路线, 不走弯路
- ✓ 企业级 AI 应用项目实战, 直接写进简历
- ✓ AI 应用开发面试宝典, 覆盖高频考点
- ✓ 1v1 简历包装, 现有项目→AI应用项目
- ✓ 1v1 模拟面试 + AI 岗投递辅导

添加导师微信,
免费评估你的 AI 转型可行性



转 AI 应用开发, 最快 2 周, 平均一个月

介绍

最近几年, AI 大模型已成为科技领域最热门的话题之一。从 ChatGPT 的爆火到国内 Deepseek、通义千问等产品的涌现, 大模型正在刷新我们对人工智能的认知。那么, 究竟什么是 AI 大模型? 它为何能在短时间内引发如此巨大的关注? 作为后端开发人员, 我们又该如何利用这一技术构建创新应用?

简单来说, AI 大模型是指通过海量数据和强大算力训练出的超大规模神经网络。这类模型一般包含数十亿甚至数万亿个参数, 能够处理和理解复杂的信息模式。以 OpenAI 的 GPT-4 为例, 其参数量高达 1.8 万亿, 训练数据规模相当于数百万本书的内容。这种复杂的模型结构, 具有非常强大的能力, 比如说理解上下文、生成连贯文本, 甚至跨模态处理图像和语言。大模型的核心优势在于其预训练+微调的范式。预训练阶段, 模型通过无监督学习从互联网规模的通用数据

中吸收知识；微调阶段则针对特定任务进行优化。这种模式打破了传统AI“一个任务一个模型”的局限，让单一模型能够灵活适应多种场景。

和传统 AI 开发不同，大模型应用开发的核心不是从头训练模型，而是通过工程化手段释放现成模型的潜力。这就像组装乐高积木——开发者需要巧妙组合Prompt工程、向量数据库、业务逻辑等模块，将通用大模型训练为特定场景的解决方案。

以个人知识库助手为例，开发者只需将用户文档转化为向量嵌入，再通过语义检索找到相关片段作为上下文，就能让 GPT-4 等模型生成精准回答。

学习建议

如果你是个后端开发者如何转向 AI 大模型应用开发呢？

想要从传统的业务开发转向 AI 应用开发。但是，大部分人对这个转型的具体路径还是比较迷茫的：要不要去学 PyTorch？是不是得先补一补机器学习和数学？后端开发的经验能不能在 AI 领域复用？

其实，从后端转 AI 应用开发，不是简单的“换个技术栈”，而是思维方式、工程经验、业务理解能力的综合迁移。

AI 应用的核心逻辑

很多人一提到 AI，就觉得是“高深的数学 + 复杂的算法 + 神秘的模型”。但如果你的目标是做 AI 应用，而不是去研究新模型，那么核心思路其实跟做后端业务没什么本质区别的：

- 1) 大模型本质上是个 **强大的 API**，它能做的事情远比普通 API 复杂，但它仍然需要业务逻辑去组织。
- 2) 你需要设计 **Prompt**（提示词工程），就像写 SQL 查询一样，要学会用“对 AI 友好的方式”让它输出符合要求的内容。
- 3) 你需要**处理上下文**、存储用户数据、结合业务规则，而这些其实是后端开发最擅长的部分。

换句话说，AI 应用开发的关键，不是研究 AI 本身，而是学会 **如何围绕 AI 设计出好用的产品**。

后端开发的优势

很多人觉得 AI 相关的工作应该由算法工程师或者机器学习专家来做，但实际上，后端开发者在 AI 应用落地上有天然的优势。

首先，后端开发者对 **系统架构、数据处理、性能优化** 都有深入的理解，而 AI 应用的落地往往需要这些能力。比如，你要做一个 AI 面试官，它需要：

- 通过 WebSocket 或者 SSE 来进行流式实时通信。
- 使用 MySQL 来存储用户的历史对话，避免每次请求都从零开始。
- 使用 TTS 来实现进行实时语音对话功能

这些需求，本质上和普通的后端开发没什么区别，只是服务的核心逻辑变成了“调用大模型”而已。

其次，后端开发者更熟悉 **业务需求和应用工程化落地**。大多数 AI 算法专家更擅长优化模型，而不是做一个真正好用的产品。能把大模型接入业务场景，并且优化体验、降低成本，这种能力是非常稀缺的。

学习目标

当然，后端开发者转 AI 还是有一些新的东西需要学习，但比想象中简单得多。

- **理解大模型的能力边界**：很多人对 AI 的能力有误解，觉得它能解决所有问题，但实际上，大模型有明显的上下文限制、幻觉问题、稳定性问题。你需要了解它的优势和局限，才能用好它。

- **掌握 Prompt 设计**：不同的提示词会影响大模型的输出，甚至决定最终产品的效果。比如 “你是谁？” 和 “你是一个 Java 面试官，请问 Spring Boot 的核心特性是什么？” 这两种问法，得到的答案完全不同。
- **熟悉 LangChain、RAG、LLM、向量数据库**：这些工具能帮助你更高效地搭建 AI 应用，比如用向量数据库（如 Milvus、Faiss）做长文本搜索，用 LangChain 处理复杂的多轮对话。

但整体来看，后端开发者完全可以 **沿着自己的经验积累** 来进入 AI 领域，而不需要完全推翻已有的知识体系。

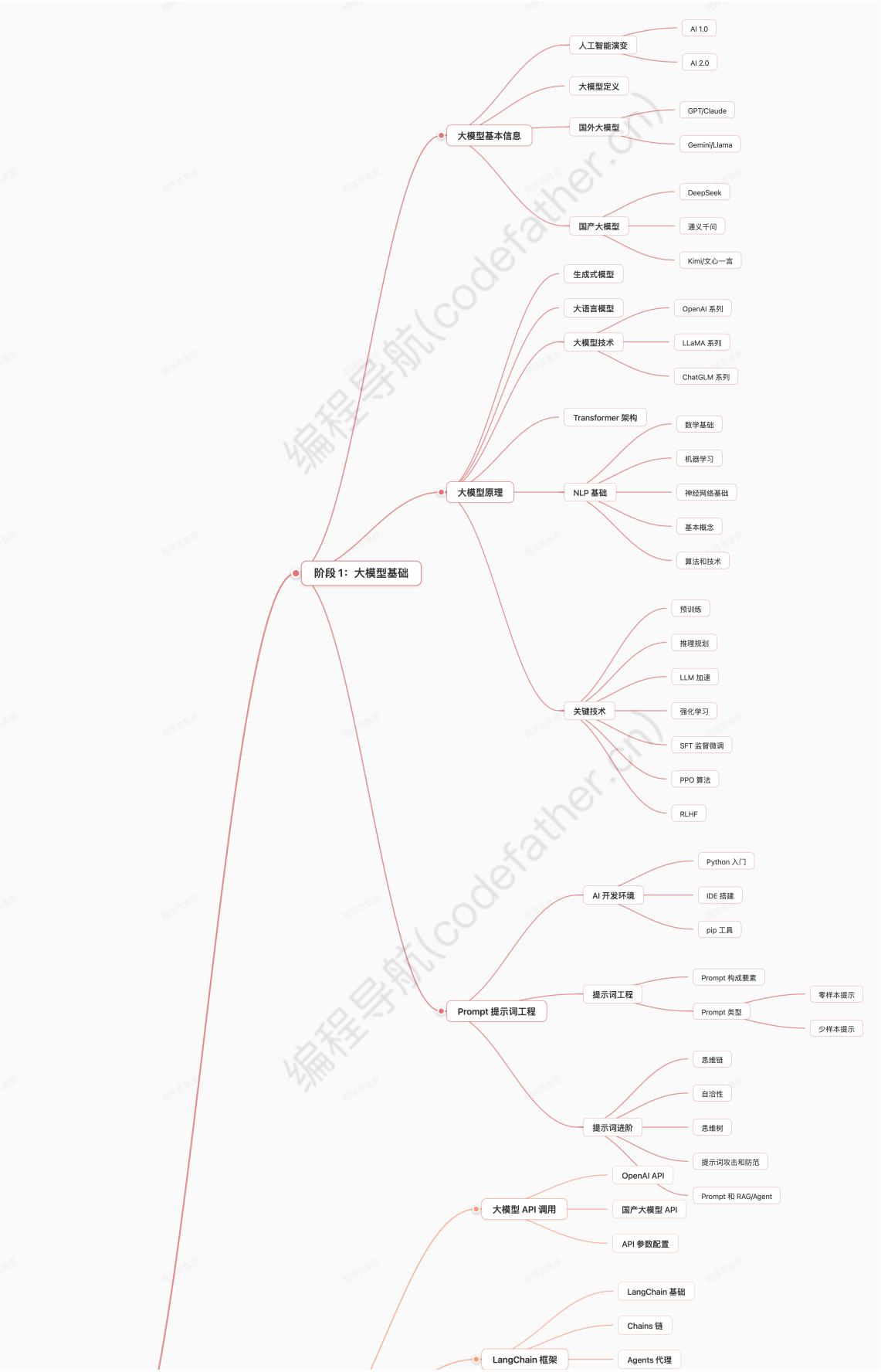
学习路线

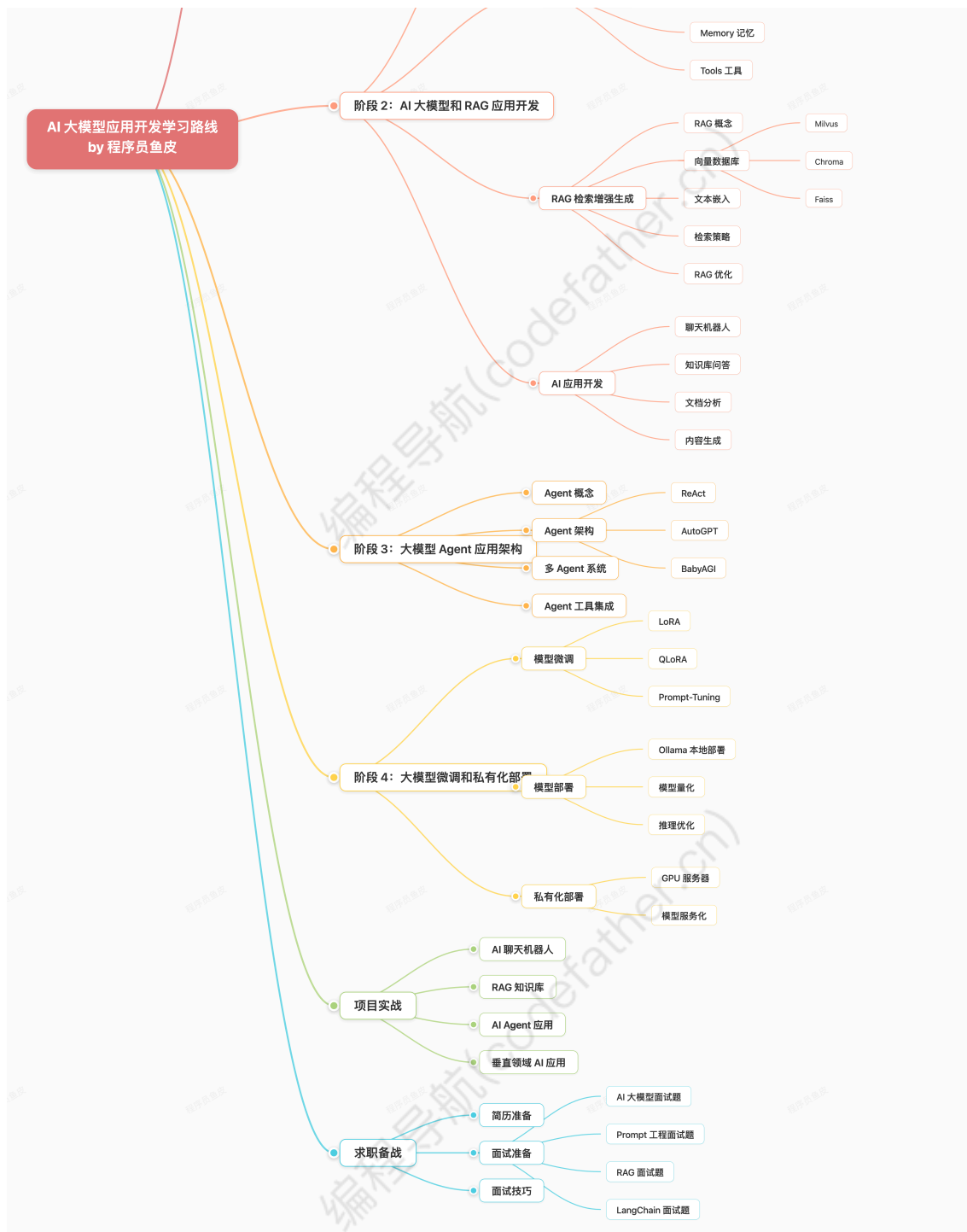
可以按照这个路线去学习：

- 1) 大模型基础
- 2) AI大模型和 RAG 应用开发工程
- 3) 大模型Agent应用架构
- 4) 大模型微调 and 私有化部署

学习过程中，推荐使用 [AI 资源导航](#) 中的工具辅助学习，用 AI 学 AI 会更高效。

学习路线图





阶段1：大模型基础

大模型的基本信息（5天）

知识

- 人工智能的演变
 - 什么是AI
 - AI 1.0

- AI 2.0
- 大模型和通用人工智能
 - [大模型的定义](#)
 - [人工智能的定义](#)
 - 大模型和人工智能的产品
- 国外大模型
 - [GPT](#)
 - [Claude](#)
 - [Gemini\(Bard\)](#)
 - [Llama](#)
 - [Gemini](#)
 - [Mistral Large](#)
 - [Grok](#)
- 国产人工智能
 - [DeepSeek](#)

- [通义千问](#)
- [豆包](#)
- [Kimi](#)
- [文心一言](#)
- [讯飞星火](#)
- [ChatGLM 系列](#)
- [百川 Baichuan](#)
- [腾讯元宝](#)

学习建议

- 打基础：可以先从一些入门书籍、视频课程开始，比如 B站上关于 AI 历史和大模型技术的课程，了解概念和背景。
- 关注动态：多关注一些技术博客和论坛，比如huggingface、GitHub上的开源模型和项目，及时了解国外和国内的最新进展。
- 动手实践：可以从简单的例子入手，比如使用ChatGPT或者DeepSeek进行对话，看看是怎么输出的。

学习资源

1) 李沐讲AI (动手学深度学习在线课程)

课程: <https://courses.d2l.ai/zh-v2/>

视频: <https://space.bilibili.com/1567748478?>

2) 马士兵-AI大模型全套教程

<https://www.bilibili.com/video/BV128cUe6EU2/>

3) AI大模型零基础全套教程

- [黑马 Python+AI 大模型零基础到项目实战](#): 比较新的视频教程
- <https://www.bilibili.com/video/BV1uNk1YxEJQ>

经典面试题 (更多面试题)

- 1) 人工智能的发展经历了哪些主要阶段? 请简要概述每个阶段的特点。
- 2) 如何理解大模型和通用人工智能 (AGI) 之间的关系? 它们在目标和实现上有何区别?
- 3) 请列举当前国际上主流的大型语言模型, 并简要介绍它们的特点和应用场景。
- 4) 在国产人工智能领域, 您了解哪些具有代表性的大模型? 它们在技术和应用上有哪些突破?
- 5) Transformer架构在大模型中扮演什么角色? 其核心机制是什么?

大模型的原理 (60天)

知识

- 理解生成式模型和大语言模型
 - 什么是[生成式模型](#)
 - 什么是[大语言模型](#)
 - 生成式模型和大语言模型的区别
- 大模型技术
 - OpenAI系列
 - [GPT1](#)
 - [GPT2](#)
 - [GPT3](#)

- InstructGPT
 - [GPT4](#)
- LLaMA系列
 - [LLaMA](#)
 - [Alpaca](#)
 - [Vicuna](#)
 - [BaiChuan](#)
 - [LLaMA2](#)
 - [BaiChuan2](#)
- BLOOM系列
 - [BLOOM](#)
 - BLOOMZ
- ChatGLM系列
 - [ChatGLM](#)
 - [ChatGLM2](#)
- Transformer架构
 - 什么是[Transformer](#)
 - 为什么会用到Transformer
 - [Transformer的架构解析](#)
- [NLP基础](#)（自然语言处理）。关于 NLP 自然语言处理的详细学习，可以查看 [NLP 自然语言处理学习路线](#)
 - 数学基础
 - 线性代数
 - 概率统计
 - 微积分
 - 最优化
 - [机器学习](#)。关于机器学习的详细学习，可以查看 [机器学习学习路线](#)
 - TF-IDF（词频-逆文档频率）
 - 朴素贝叶斯
 - SVM在文本分类中的应用
 - 线性回归
 - 梯度下降
 - 逻辑回归
 - Softmax
 - 神经网络基础。关于深度学习的详细学习，可以查看 [深度学习学习路线](#)
 - RNN/LSTM（时序建模）
 - CNN（文本分类）
 - Optimizer优化器
 - PyTorch 深度学习框架
 - CNN 卷积神经网络
 - 了解基本概念
 - 分词
 - 文本预处理
 - 特征提取
 - 语义角色标注
 - 命名实体识别
 - 算法和技术

- TF-IDF (词频-逆文档频率)
 - Word2Vec (词向量)
 - BERT (双向编码器表示)
 - GloVe (全局向量词嵌入)
 - ELMo (深度上下文词表示)
 - NLP模型
 - 循环神经网络 (RNN)
 - 长短期记忆网络 (LSTM)
 - 注意力机制
 - NLP任务
 - 文本分类
 - 情感分析
 - 机器翻译
 - 文本摘要
- 关键技术
- 预训练
 - 预训练模型中的分词器
 - BPE详解
 - WordPiece详解
 - Unigram详解
 - SentencePiece详解
 - 分布式训练
 - 分布式训练概述
 - 分布式训练并行策略
 - 分布式训练的集群架构
 - 推理规划
 - 思维链提示 (Chain-of-Thought Prompting)
 - 论文
 - 实战
 - 由少至多提示 (Least-to-Most Prompting)
 - LLM加速
 - 注意力优化
 - FlashAttention (闪电注意力)
 - PagedAttention (分页注意力)
 - CPU推理加速
 - 推理优化框架
 - 强化学习
 - 强化学习概述
 - 强化学习环境
 - 强化学习算法
 - Q-learning算法
 - DQN算法
 - Policy Gradient算法
 - Actor-Critic算法
 - SFT (监督微调, Supervised Fine-Tuning)
 - [PPO](#) (近端策略优化算法)

- PPO介绍
- 广义优势估计
- PPO算法原理剖析
- PPO算法对比
- 基于PPO的正向情感倾向性
- [RLHF](#) (基于人类反馈的强化学习, Reinforcement Learning from Human Feedback)
 - InstructGPT模型分析
 - RLHF论文
 - RLHF的流程
 - RLHF内部剖析
 - 数据收集和模型训练
 - 数据预处理模块
 - 模型训练\生成\评估
 - MOSS-RLHF
 - 奖励模型训练
 - PPO 微调

学习建议

- 夯实基础：无论学习什么，基础是最重要的，可以从自然语言处理（NLP）的基础知识开始，包括数学基础、机器学习和神经网络等概念。理解这些基础知识将有助于你更好地掌握后面的内容，也能在后面解决问题时节省很多时间。但是也不要死磕基础知识的原理，不用去卷数学证明或底层推导，只需要掌握理解模型结构所需的知识点，知道最里面的算法和流程，能看懂主流模型是怎么工作的，怎么训练和使用的，除非你想造模型，大模型的应用开发更重工程能力。
- 理解核心概念：深入学习生成式模型、大语言模型以及Transformer架构，掌握预训练、推理规划、强化学习等关键技术。
- 关注实践：理论结合实践，尝试在本地部署小型模型，完成简单的文本生成任务，以加深对理论的理解。

学习资源

1) NLP 新手入门教程

<https://github.com/PKU-TANGENT/nlp-tutorial>

2) Transformers 教程

<https://transformers.run/>

3) 深入理解Transformer技术原理

<https://tech.dewu.com/article?id=109>

4) Transformer 是如何工作的

<https://huggingface.co/learn/llm-course/zh-CN/chapter1/4>

5) HuggingFace快速入门

<https://huggingface.co/docs/transformers/quicktour>

6) 动手学大模型应用开发

<https://datawhalechina.github.io/llm-universe/#/>

7) 基于transformers的自然语言处理(NLP)入门

<https://datawhalechina.github.io/learn-nlp-with-transformers/#/>

8) Ollama本地模型部署

<https://ollama.ai/>

9) LLM/AI 大模型入门指南

<https://zhuanlan.zhihu.com/p/722000336>

经典面试题 (更多面试题)

- 1) [说说你了解的机器学习是什么?](#)
- 2) [监督学习、半监督学习和无监督学习分别是什么, 它们的区别在哪?](#)
- 3) [什么是词嵌入 \(Word Embedding\)? 有哪些常见的词嵌入方法?](#)
- 4) [什么是深度学习? 它与传统机器学习有什么区别?](#)
- 5) [如何评估一个深度学习模型的性能?](#)

Prompt (10天)

知识

- AI 开发环境
 - Python 快速入门
 - IDE 环境的搭建
 - pip 工具的安装和使用
- 提示词工程
 - 提示词和 (Prompt) 提示工程
 - Prompt的典型构成要素
 - Prompt的类型
 - 零样本提示
 - 少样本提示
 - 上下文学习
- 提示词工程进阶
 - Prompt 调优进阶技巧
 - 思维链 (Chain-of-thought)
 - 自洽性 (Self-Consistency)
 - 思维树 (Tree-of-thought)
 - 提示词攻击和防范
 - 提示词注入
 - 提示词防范
 - 提示词的未来展望
 - Prompt和RAG的关系
 - Prompt和Agent的关系
 - Prompt和微调的关系

学习建议

学习AI免不了要跟Python打交道, 因为AI的很多库和框架都是Python写的, 所以我们首先要学好Python。关于Python 的详细学习, 可以查看 [Python 学习路线](#)

对于提示词工程，这部分其实是大模型应用中一个很有意思的环节。你需要先搞清楚什么叫Prompt，了解它的概念、作用，以及如何通过设计有效的提示词来引导大模型生成期望的输出。关于 Prompt Engineering 提示词工程的详细学习，可以查看 [Prompt Engineering 提示词工程学习路线](#)

然后来再一步步试着调试和优化Prompt的效果，这个过程很大程度上依赖于不断地实践和尝试，所以一定要动手尝试，才能将所学知识应用于实际问题，强化理解。

学习资源

1) Python学习路线

关于 Python 的详细学习，可以查看 [Python 学习路线](#)

2) OpenAI官方Prompt工程指南

<https://platform.openai.com/docs/guides/prompt-engineering>

3) Prompt模板库

<https://github.com/f/awesome-chatgpt-prompts>

4) 提示词工程指南

<https://www.promptingguide.ai/zh>

<https://github.com/dair-ai/Prompt-Engineering-Guide>

经典面试题 [\(更多面试题\)](#)

- 1) [什么样的 prompt 是好的 prompt?](#)
- 2) [你给 AI 预设的 Prompt 结构是什么样的? 如何优化 Prompt](#)
- 3) 什么是提示词攻击? 有哪些常见的防范措施可以提高模型的安全性?
- 4) 请解释提示词的不同类型，并讨论它们在实际应用中的适用场景。

大模型 API (3天)

知识

- 概念介绍
- 专业术语
 - Endpoints
 - Token
 - Prompt
 - APIKey
- 提示词 (Prompt) 优化
- 流式输出 (WebSocket、SSE)
- Token 计算

学习建议

调大模型API其实跟调用普通的接口是一样的，了解其中的输入输出参数，调用方法，不同的是，如果想要实时的输出，需要通过SSE或者WebSocket的方式来模拟打字机的效果。另外，你需要知道的是，大模型API里有一个比较重要的概念，那就是token，需要知道它是怎么计算的，因为它是跟钱相关的，涉及到费用计算。然后就是多去实践，现在很多网站注册都会送很多的token，可以去调用试一下效果。

学习资源

1) OpenAI的API文档

<https://platform.openai.com/docs/api-reference/introduction>

2) OpenAI 的token计算器

<https://platform.openai.com/tokenizer>

3) Deepseek的API文档

<https://api-docs.deepseek.com/>

经典面试题 (更多面试题)

- 1) 什么是Token? 在大模型API中, 如何计算输入和输出的Token数量?
- 2) 流式输出在大模型API中有哪些实现方式? 请比较WebSocket和SSE的特点及适用场景。
- 3) 在使用大模型API时, 如何确保API密钥的安全性? 请说明常用的安全措施。

阶段2: RAG应用开发工程

RAG (15天)

关于 RAG 检索增强生成的详细学习, 可以查看 [RAG 检索增强生成学习路线](#)

知识

- 检索增强生成
 - LLM的缺陷分析
 - [RAG的定义](#)
 - RAG应用场景分析
 - RAG的三大范式
 - Naive RAG
 - Advanced RAG
 - Modular RAG
 - RAG的三大部件
 - 检索器 Retriever
 - 生成器 Generator
 - 增强方法 Augmentation Method
 - Naive RAG Pipeline (朴素检索增强生成流程)
 - 知识库构建
 - 文档加载和分块
 - 分块方案详解
 - Embeddings 向量化
 - 向量化的意义
 - 向量化实践
 - OpenAI Embedding模型
 - 百度文心Embedding-V1
 - GLM Embedding模型

- bge-large-zh-v1.5
- 向量相似度算法
 - 余弦距离Cosine
 - 点积IP
 - 欧式距离L2
- 向量数据库
 - 向量数据库的作用
 - 向量数据库类型
 - 主流向量数据库和功能对比
- Prompt上下文增强设计

学习建议

RAG 现在挺火的，它本质上就是让大模型先查资料再回答问题，比直接训练模型要容易上手。你可以从了解它的基本概念开始，先思考一下为什么需要检索增强，不理解其中的技术细节也没关系，可以先从一个项目例子入手，比如让系统先从某个文档集中找到相关内容，再将这些内容作为输入传递给生成模型，看看它能如何生成更准确的答案。在实践的过程中可以看看它是怎么运行的，了解下 RAG 的流程。然后可以着重理解RAG的应用场景和它在实际项目中的工作原理，再逐步深入学习如何调优检索部分和生成部分的协同工作，如何评估结果，并做性能优化，并且可以了解下里面用到的技术。

学习资源

1) Hugging Face的RAG文档，详细介绍了如何实现和应用RAG

https://huggingface.co/docs/transformers/main/en/model_doc/rag

2) 基于 RAG 的知识库管理

<https://github.com/rag-web-ui/rag-web-ui>

3) RAG教程

<https://www.bilibili.com/video/BV1hccSeJEUD>

4) 用 Hugging Face Zephyr 和 LangChain 针对 Github issues 构建简单的 RAG

https://huggingface.co/learn/cookbook/zh-CN/rag_zephyr_langchain

5) 手把手带你结合企业级项目实战完成一套完整的RAG项目

<https://www.bilibili.com/video/BV1YaRhY9EqV>

经典面试题 [\(更多面试题\)](#)

- 1) [什么是 RAG? RAG 的主要流程是什么?](#)
- 2) [什么是 RAG 中的 Rerank? 具体需要怎么做?](#)
- 3) [当发现RAG系统召回结果与用户query意图不匹配时, 有哪些可能的改进方向?](#)
- 4) [如何评测大模型的幻觉?](#)

RAG的三大范式和优化 (10天)

知识

- 三大范式
 - Naive RAG

- 索引
 - 检索
 - 生成
- Advanced RAG
 - 预检索
 - 检索增强
 - 后检索
- Modular RAG
 - 混合检索
 - 智能编排
 - 技术融合
- RAG技术的变体
 - T-RAG (时序感知RAG)
 - CRAG (纠错增强RAG)
 - Self-RAG (自反思RAG)
 - RAG-Fusion (多查询融合RAG)
 - Rewrite-Retrieve-Read RAG (查询重写RAG)
- RAG分析和优化方案
 - RAG分析
 - 文档加载准确性和效率
 - 文档切分的粒度
 - 错过排名靠前的文档
 - 提取上下文和答案无关
 - 格式错误
 - 答案不完整
 - 未提取到答案
 - 答案太具体或太笼统
 - 幻觉问题
 - 索引优化
 - 改善数据细节度
 - 优化索引结构
 - 元数据索引原理
 - 摘要索引原理
 - 父子索引原理
 - 假设性问题索引原理
 - 检索前优化
 - 微调Embedding模型
 - 混合检索
 - 问题转换
 - 检索后优化
 - 召回重排
 - 信息压缩
 - 知识融合

学习建议

这一阶段主要是深入学习RAG的一些优化技术和设计，如三大范式等。可以重点关注 RAG 的核心机制，如检索、生成、索引优化、检索前后优化等。深入理解这些概念，有助于在实际应用中灵活运用。可以多看一些网上的文章，这样有助于理解。

学习资源

1) 一文读懂：大模型RAG（检索增强生成）含高级方法

<https://zhuanlan.zhihu.com/p/675509396>

2) RAG 技术的发展历程

<https://developer.aliyun.com/article/1597639>

3) RAG 优化方案和实践

<https://zhuanlan.zhihu.com/p/703182970>

4) RAG 范式、技术和趋势

<https://www.cnblogs.com/xiaoqi/p/18075992/rag-survey>

经典面试题（更多面试题）

- 1) 请比较Naive RAG、Advanced RAG和Modular RAG三种范式的主要区别，以及它们各自适用的应用场景。
- 2) 在RAG技术的变体中，T-RAG、CRAG和Self-RAG分别解决了哪些特定问题？
- 3) 在RAG系统的索引优化过程中，如何利用元数据索引、摘要索引和父子索引来提升检索效率？
- 4) 针对RAG系统中常见的幻觉问题，你会采用哪些分析和优化方案来减少其发生？
- 5) 在检索前优化阶段，微调Embedding模型和混合检索各有哪些优势？

RAG项目评估（5天）

知识

- RAG效果评估
 - 质量指标
 - 上下文相关性
 - 答案忠实度
 - 答案相关性
 - 能力指标
 - 对噪声的鲁棒性
 - 负面信息的排除能力
 - 信息整合能力
 - 评估工具
 - [RAGAS](#)
 - [ARES](#)
 - [Trulens](#)

学习建议

如何评估RAG项目的效果是确保模型性能和提升用户体验的关键一步。在这一阶段，你需要掌握质量指标、能力指标以及评估工具的使用方法。重点在于理解如何衡量模型生成答案的准确性、相关性，以及检索内容的质量。可以利用评估工具对RAG系统进行全面分析，从而为模型的优化提供数据支持。

学习资源

1) 使用合成数据和 LLM 作为裁判评估 RAG

https://huggingface.co/learn/cookbook/zh-CN/rag_evaluation

2) 一次搞懂RAG评估，三个角度LangChain, LlamaIndex, RAGAS

<https://www.bilibili.com/video/BV1aZ421W7DB/>

经典面试题 [\(更多面试题\)](#)

- 1) 请解释在RAG系统中，答案的“忠实度”和“相关性”有何不同？在实际项目评估中，如何分别度量这两个指标？
- 2) 你如何评估一个RAG系统是否具备良好的信息整合能力？
- 3) 在评估RAG系统的鲁棒性时，你会设计哪些实验来检测其对噪声的容忍能力？
- 4) 一个RAG系统频繁生成带有负面信息的回答，你会如何定位问题并使用哪些指标或工具进行优化？

RAG 项目 (7天)

知识

- [RAGFlow](#)
- [FastGPT](#)
- [QAnything](#)
- [LangChain-chatchat](#)
- [GraphRAG](#)

学习建议

前面学习的都是RAG的理论知识，下面可以深入去实践RAG项目了，通过实际的开源项目，可以将理论知识应用于实践，深化对 RAG 技术的理解。通过阅读这些项目的源码，可以了解不同 RAG 框架的实现细节和设计思路。建议选择一个和自己目标最契合的项目，深入研究其架构和代码实现，并尝试进行本地部署和测试。

阶段3：大模型Agent应用架构

LangChain (15天)

关于 LangChain 开发的详细学习，可以查看 [LangChain 开发学习路线](#)

知识

- [什么是LangChain](#)
- LangGraph提示词管理，监控，链定义和管理
- LangChain和Tavily检索
- LangChain的核心组件
 - Chat models VS LLMs
 - 流式输出
 - token 追踪
 - 结构性输出
 - 模型I/O封装
 - Prompts模版
 - 自定义Prompts模版
 - 序列化模版
 - 数据连接
 - 文本向量化实现方式
 - 向量数据库
 - Chroma
 - ES

- FAISS
- Milvus
- 文档转换切割
- Memory记忆封装
 - 内置链
 - Memory 工具使用
 - 为链增加Memory
 - 多轮对话的历史记录
- 链 (chain)
 - LCEL表达式
 - LCEL Runnable 协议设计和使用
 - LCEL 进阶使用：整合复杂逻辑的多链
 - LCEL添加记忆
 - LCEL中chain 和 prompt结合

学习建议

学习 LangChain 框架对于大模型应用开发非常重要。它是一个开源框架，它简化了和大型语言模型（LLM）的集成，可以帮助你快速构建复杂的 AI 应用。你可以先了解 LangChain 的核心概念，如提示词管理、链的定义和管理等，这些是构建应用的基础。接着，深入学习 LangChain 的核心组件，包括 Chat Models 和 LLMs 的区别、模型 I/O 封装、数据连接、Memory 记忆封装以及链（Chain）的使用。这些组件是 LangChain 的精髓，掌握它们将使你能够灵活地构建和优化应用。学习过程中，不用死磕底层，先跑通几个Demo（比如本地知识库问答），再慢慢深入。学到能独立用 LangChain接API、处理数据、搭出可用的AI工具就够用了，细节可以边做边补。

学习资源

1) LangChain官方文档（Python）

<https://python.langchain.com/docs/introduction/>

2) LangChain中文文档

<https://www.langchain.com.cn/>

3) LangChain大模型全套教程

<https://www.bilibili.com/video/BV1BgfBYoEpQ>

4) LangChain 教程

<https://www.langchain.asia/>

5) LangChain 的中文入门教程 Topics

<https://github.com/liaokongVFX/LangChain-Chinese-Getting-Started-Guide>

经典面试题（更多面试题）

- 1) LangChain 是什么？它的核心功能和特点是什么？请简要解释它如何帮助构建基于大语言模型的应用。
- 2) [解释LangChain框架中的Chain和Agent概念，并举例说明各自的应用场景](#)
- 3) [使用LangChain时，如何实现多路召回结果的动态权重分配？](#)
- 4) [请描述使用LangChain构建一个文档问答系统的关键技术组件及实现步骤](#)
- 5) LangChain 中的 Memory 组件如何工作？它如何帮助大语言模型保持上下文并支持多轮对话？

LlamaIndex (5天)

知识

- LlamaIndex 是什么
- LlamaIndex 的优势和劣势
- LlamaIndex 和 RAG 检索增强联合应用实践
- LlamaIndex 和 LangChain 对比分析

学习建议

学习LlamaIndex 能帮助你将LLM与特定领域的数据相结合，构建更智能的应用程序。可以先读一些文档，了解LlamaIndex的基本概念和使用模式。然后，通过实践教程，尝试搭建一个简单的文档问答系统，加深对其核心组件的理解。

学习资源

1) LlamaIndex 官方文档

<https://docs.llamaindex.ai/en/stable/>

2) LlamaIndex 使用指南

<https://llama-index.readthedocs.io/zh/latest/guides/primer.html>

3) LlamaIndex零基础全套课程

<https://www.bilibili.com/video/BV1JDpFeEEay/>

4) LlamaIndex 教程

<https://llama-index.readthedocs.io/zh/latest/guides/tutorials.html>

经典面试题 [\(更多面试题\)](#)

- 1) LlamaIndex 是什么，它的主要功能和目标是什么？
- 2) LlamaIndex 如何构建文档索引？你能描述一下它的索引结构吗？
- 3) 在 LlamaIndex 中，如何处理和管理大量非结构化数据？
- 4) 在 LlamaIndex 中，如何对文档进行预处理，以确保索引的效率和检索质量？
- 5) LlamaIndex 如何处理复杂的查询和多步骤推理任务？

Agent (20天)

关于 AI Agent 应用开发的详细学习，可以查看 [AI Agent 应用开发学习路线](#)

知识

- Agents关键技术分析
 - Agents 介绍
 - Agents流程、决策图
 - 规划 (Planning)
 - 子任务拆解
 - 反思和改进
 - 记忆 (Memory)
 - 工具使用 (Tools)
 - 预制工具 (Tool)
 - 预制工具集 (Toolkits)

- 自定义工具
 - 执行 (Action)
- Function Calling
 - Function Calling的诞生背景
 - 如何理解Function Calling
 - Function Calling的实现过程
 - 远程Function Calling调用
 - 支持Function Calling的国产模型
- Agent认知框架
 - ReAct思考-行动-观察
 - Plan-and-Execute
 - Self-Ask
 - Thinking and Self-Reflection
- 多Agent系统
 - AutoGPT快速打造智能体
 - CAMEL策略
 - AutoGen
 - MetaGPT

学习建议

学习Agent能让大模型从“只会聊天”变成“能干活”的智能助手。建议从LangChain这类框架入手，先写个能查天气或订票的Demo，再慢慢加复杂功能（比如多Agent协作）。学到能独立设计一个能自动完成任务的Agent就够用了，比如客服自动回复系统或者数据分析助手等等。

学习资源

1) Agent教程

<https://github.com/datawhalechina/agent-tutorial>

经典面试题 [\(更多面试题\)](#)

- 1) 什么是Agent？在AI应用中，Agent的核心作用是什么？
- 2) Agent系统中，如何进行子任务拆解和反思？它们如何帮助提高决策效率？
- 3) Agent中的记忆(Memory)如何作用于系统的学习和决策？
- 4) 在Agent系统中，如何实现远程Function Calling？有哪些常见的技术实现方式？
- 5) 如何通过规划和执行(Plan-and-Execute)来优化Agent的任务执行效率？

可视化开发框架/Agent IDE (10天)

知识

- GPTs
 - [GPTs](#)
 - Assistants API
 - 应用场景
 - Assistants thread和messages
 - 原生API

- 开源大模型
- Coze扣子
 - [Coze](#)
 - Coze基础
 - Coze概述
 - AI Agent的人设
 - AI Agent插件系统
 - AI Agent工作流
 - 知识库、数据库和变量-AI Agent的记忆
 - AI Agent发布
 - Coze实操
 - 构建知识库
- Dify 开源的应用编排工具
 - [Dify](#)
 - Dify基础架构
 - Dify实操

学习建议

学习完AI Agent 之后就可以实操了，这些工具能让你不用写太多代码就能快速搭建AI应用。可以先了解这3个框架的特点，然后可以使用他们搭建个AI应用。

学习资源

- 1) 【教程】快速上手 OpenAI GPTs：分分钟创建你的专属 GPT

<https://www.bilibili.com/video/BV1gG411X7q7/>

- 2) Dify搭建简单的知识库问答工作流

<https://www.bilibili.com/video/BV1M29PYiEHx/>

- 3) Dify官方文档

<https://docs.dify.ai/zh-hans>

经典面试题 [\(更多面试题\)](#)

- 1) 在使用Dify进行开发时，如何进行应用编排？简要描述Dify的实操步骤。
- 2) 如何使用Assistants API来构建AI助手
- 3) 如何定义AI Agent的人设？在人设设计中，AI Agent的任务和目标如何影响其行为？
- 4) 在Coze中，如何构建和管理知识库？它如何支持AI Agent的记忆功能？

阶段4：大模型微调和私有化部署

Transformer（10天）

知识

- Transformer结构理解
- Transformer 模型总体架构
- 理解Self-Attention
- 理解Encoder 和 Decoder
- Multi-head Attention

- 不同Decoding方法

学习建议

学习大模型应用开发，Transformer是必须跨过的坎儿，因为它是所有大模型的"心脏"。重点先搞懂三个核心：自注意力机制（为啥模型能同时关注所有词）、编码器-解码器结构（怎么把输入变成输出）、位置编码（为啥不用RNN也能记住顺序）。建议边学边动手，比如用PyTorch从头实现一个迷你Transformer（不用太复杂，能跑通文本分类就行），再对比Hugging Face的现成API感受差异。学到能说清楚BERT和GPT的区别、能自己调通一个微调任务就够用了。

学习资源

- 1) 强烈推荐！台大李宏毅自注意力机制和Transformer详解！

<https://www.bilibili.com/video/BV1v3411r78R>

- 2) Transformer从零详细解读

<https://www.bilibili.com/video/BV1Di4y1c7Zm>

- 3) Transformer 快速入门

<https://transformers.run/>

- 4) 图解Transformer

<https://zhuanlan.zhihu.com/p/347904940>

- 5) Transformer 使用教程

<https://huggingface.co/docs/transformers/zh/quicktour>

经典面试题 (更多面试题)

- 1) [聊一聊 Transformer 的架构和基本原理。](#)
- 2) [Transformer 的哪个部分最占用显存？](#)
- 3) [讲一下你对 Transformer 的 Encoder 模块的理解](#)
- 4) [Transformer 中，Decoder 阶段的多头自注意力和 Encoder 阶段的多头自注意力是相同的吗？](#)
- 5) [Transformer 和 LLM 有哪些区别](#)

开源模型 (20天)

知识

- 国外开源模型和框架
 - [Llama](#)
 - [Falcon](#)
 - [vLLM](#)
 - [OpenLLM](#)
 - [Ollama](#)
 - [Mistral](#)
- 国内开源模型
 - [ChatGLM](#)
 - [Qwen](#)
 - [Deepseek](#)
 - [Baichuan](#)

- [Moss](#)
- [InternLM](#)

学习建议

可以先搞懂几个主流模型的特点，然后本地部署一下，可以优先把主流的国外模型和国内模型都跑通一遍，比如用Ollama跑个Llama3、用vLLM部署一个ChatGLM，重点是理解它们的模型结构、推理速度优化策略、部署环境依赖这些工程细节，未来你在做模型服务端开发、评估切换模型时才不会一头雾水。

学习资源

1) llama3 视频教程

<https://www.bilibili.com/video/BV1bD62YdE9A>

2) 部署Ollama教程

<https://www.bilibili.com/video/BV13e1jY9EmZ/>

3) 动手学Ollama

<https://datawhalechina.github.io/handy-ollama/#/>

4) Dify 模型接入

<https://docs.dify.ai/zh-hans/development/models-integration>

Fine-Tuning（模型微调）（15天）

知识

- [模型微调介绍](#)
- 如何选择合适的基座模型
- 数据集的收集和预处理
 - 数据增强
 - 数据清洗
 - 数据去重
- 微调训练框架
 - [Hugging Face Transformers](#)
 - [PyTorch](#)
 - [DeepSpeed](#)

学习建议

开源模型的微调是比较关键的，重点先搞懂三件事：选基座模型（中文任务优先选ChatGLM3或Qwen，英文选Llama3）、处理数据（清洗比增强更重要，别让脏数据带偏模型）、跑通微调流程（先用Hugging Face Transformers+LoRA试手，再尝试DeepSpeed加速）。建议从简单的分类任务开始，比如用ChatGLM3微调一个工单分类器，数据集不用大，几百条高质量数据就能看到效果。学到能独立完成一个业务场景的微调就够用了。

学习资源

1) Transformer 使用教程

<https://huggingface.co/docs/transformers/zh/quicktour>

2) deepSpeed 教程

<https://deepspeed.org.cn/getting-started/>

3) Pytorch 教程

<https://www.runoob.com/pytorch/pytorch-tutorial.html>

经典面试题 [\(更多面试题\)](#)

- 1) [请解释大模型微调\(Fine-tuning\)的原理，并说明在什么业务场景下需要微调而不是直接使用基础模型？](#)
- 2) [请解释如何使用 TensorFlow 或 PyTorch 构建和训练一个深度学习模型。](#)
- 3) [了解哪些大模型微调技术？](#)
- 4) 在进行Fine-Tuning时，如何选择适合的预训练模型？
- 5) 如何在微调过程中使用不同的学习率策略？

PEFT fine-tuning (参数高效微调) (20天)

知识

- PEFT fine-tuning 介绍
- PEFT 主流技术
 - Adapter Tuning
 - Prompt Tuning
 - Prefix Tuning
- LoRA 低秩适配微调
 - 算法原理
 - 性能效果
- LoRA 的改进和扩展
 - AdaLoRA自适应权重矩阵微调算法解析
 - QLoRA量化低秩适配微调算法解析
 - LongLoRA长上下文低秩适配微调算法解析
 - SLoRA低秩适配微调算法解析
- P-Tuning V2
 - 原理介绍

学习建议

这个阶段主要是让你在不需要庞大计算资源的情况下，对大模型进行高效微调，适应特定任务。首先需要理解PEFT的主要技术，如Adapter Tuning、Prompt Tuning和Prefix Tuning，掌握它们的原理和适用场景。接着，深入学习LoRA (Low-Rank Adaptation)，这是目前最通用、效果最好的微调方法之一。此外，可能需要了解LoRA的改进和扩展，如AdaLoRA、QLoRA、LongLoRA和SLoRA，它们在不同场景下提供了更灵活的微调策略。这个阶段有很多的专业名词，会有写晦涩难懂，需要在网上看一些文档，然后再结合实际项目去理解，这样会容易点。

学习资源

1) Hugging Face PEFT官方文档

<https://huggingface.co/docs/peft/en/index>

2) LoRA实战教程

<https://github.com/hiyouga/LLaMA-Factory>

3) 微调方案介绍

<https://datawhaler.feishu.cn/wiki/TyQZw9lZSiN1V5kTKlocgJOvnag>

4) 大模型参数高效微调技术原理和实战

<https://github.com/SwordHG/LLM-PEFT->

经典面试题 [\(更多面试题\)](#)

- 1) [LoRA 的原理了解吗?](#)
- 2) [聊聊你对 Q-LoRA \(Quantized Low-Rank Adaptation\) 的理解。](#)
- 3) 什么是参数高效微调 (PEFT)，它与传统的Fine-Tuning有何区别?
- 4) 在PEFT中，如何选择要微调的参数或模块?
- 5) PEFT在处理大规模预训练模型时如何优化内存和计算消耗?

Quantization (量化) (10天)

知识

- 模型显存占用和量化技术
 - 微调中GPU算力测算算法
 - GPU芯片性能分析
 - 模型运行算力要求
- Transformers 原生支持的大模型量化算法
 - PTQ: 训练后量化
 - QAT: 量化感知训练
- AWQ: 激活感知权重量化算法
- GPTQ: 专为 GPT 设计的模型量化算法

学习建议

量化在大模型开发学习中也是比较重要的，通过量化技术，可以显著减少模型的内存占用和计算需求，同时尽量保持模型的性能。建议可以从理解模型量化的基本概念开始，掌握训练后量化 (PTQ) 和量化感知训练 (QAT) 的原理，了解它们各自的适用场景和优缺点。接着，深入研究一些先进的量化算法，如GPTQ和AWQ，理解它们如何在保持模型精度的同时进一步压缩模型大小。重点在于理解这些算法的核心思想和实现方式，以及它们在实际应用中的效果。可以通过阅读相关论文和实践代码示例，你可以更好地掌握这些技术。

学习资源

1) 量化感知训练 (Quantization-aware-training) 探索-从原理到实践

<https://zhuanlan.zhihu.com/p/548174416>

2) AWQ量化技术解析

<https://zhuanlan.zhihu.com/p/681578090>

3) GPTQ官方实现

<https://github.com/IST-DASLab/gptq>

4) 吴恩达《深入模型量化|Quantization in Depth》

<https://www.bilibili.com/video/BV1kw4m1X7Bi/>

经典面试题 [\(更多面试题\)](#)

- 1) 什么是模型量化，为什么在深度学习中需要使用量化技术？
- 2) 量化如何影响深度学习模型的显存占用和计算效率？
- 3) PTQ（训练后量化）和 QAT（量化感知训练）有什么区别？
- 4) 量化技术对GPU芯片性能的影响有哪些？
- 5) 如何选择适合的量化算法，PTQ 和 QAT 在不同场景下的适用性如何？

语言模型训练数据 (5天)

知识

- 数据来源
 - 通用数据
 - 专业数据
- 数据处理
 - 低质过滤
 - 冗余去除
 - 隐私消除
- 数据影响分析
 - 数据规模影响
 - 数据质量影响
 - 数据多样性影响
- 开源数据集
 - Pile
 - ROOTS
 - RefinedWeb
 - SlimPajama

学习建议

在大模型中，数据的质量、来源和多样性直接影响模型效果，这个阶段你需要学习如何从通用和专业领域收集高质量的数据，掌握数据清洗、去重和隐私保护等预处理技术。此外，了解数据规模、质量和多样性对模型训练的影响，也要了解一些开源数据集的特点和应用场景。

学习资源 [\(更多面试题\)](#)

- 1) RefinedWeb数据集论文（含清洗方法）

<https://arxiv.org/abs/2306.01116>

- 2) SlimPajama数据集介绍及其处理方法

<https://huggingface.co/datasets/cerebras/SlimPajama-627B/blob/main/README.md>

- 3) 大模型预训练中文语料清洗及质量评估

https://github.com/jiangnanboy/llm_corpus_quality

经典面试题

- 1) 什么是通用数据和专业数据，它们在训练语言模型时的作用有什么区别？

- 2) 数据质量如何影响语言模型的性能，如何进行低质数据的过滤？
- 3) 如何处理训练数据中的噪声（如无意义文本或乱码）？具体有什么技术或方法？

大语言模型评估 (5天)

知识

- 模型评估介绍
- 大语言模型评估体系
 - 知识和能力
 - 伦理和安全
 - 垂直领域评估
- 大语言模型评估方法
 - 评估指标
 - 评估方法
- 大语言模型评估实践
 - 基础模型评估
 - SFT/RL 模型评估

学习建议

这个阶段你需要理解模型评估的基本概念，包括质量指标（如BLEU、ROUGE、METEOR等）和能力指标（如准确率、召回率、F1分数等），以及如何使用这些指标来衡量模型的表现。此外，熟悉常用的评估工具和框架，如GLUE、SuperGLUE等，这会有助于你在实践中有效地评估模型。

学习资源

- 1) Hugging Face评估教程

<https://huggingface.co/docs/evaluate/index>

- 2) 大模型安全评测

<https://github.com/thu-coai/Safety-Prompts>

经典面试题 [\(更多面试题\)](#)

- 1) 大语言模型评估的主要维度是什么？如何平衡这些维度的评估？
- 2) 什么是 SFT（监督微调）和 RL（强化学习）模型的评估方法？它们的评估标准有何不同？
- 3) 在评估大语言模型时，知识和能力的评估如何进行？有哪些常见的评估指标？

Multimodal (多模态) (20天)

知识

- [什么是多模态模型](#)。关于计算机视觉的详细学习，可以查看 [计算机视觉学习路线](#)
- 多模态的应用场景
- AIGC是什么
- AIGC的应用场景
- 图像生成算法
- 基于扩散模型的算法
- [DALI-3](#)与[Midjourney](#)
- [Stable Diffusion](#)与[ControlNet](#)
- 语音生成技术

- 主流TTS技术

学习建议

多模态（Multimodal）技术在大模型应用开发中至关重要，因为它使模型能够处理和整合多种数据类型，如文本、图像、音频和视频，拓宽了应用场景的广度。重点先搞懂多模态的核心概念（比如CLIP、BLIP这类模型如何对齐不同模态的数据）以及应用场景，再动手跑通一个端到端的多模态应用流程，比如用Hugging Face的Transformers库实现一个简单的图文匹配demo，能独立完成一个业务场景的多模态应用。然后可以了解一些算法，比如基于扩散模型的算法等，知道他们的原理和技术细节，可以多看看论文和解析，这样也能更快理解。

学习资源

- 1) 多模态大模型原理

<https://www.bilibili.com/video/BV1kT411o7a6/>

- 2) Stable Diffusion全套教程

<https://github.com/ai-vip/stable-diffusion-tutorial>

- 3) Midjourney入门使用教程

<https://x1d5ow2zb0f.feishu.cn/wiki/TOFnwt4lwiA4R1kMGQgcwbkqn6d>

- 4) 扩散模型详解

<https://zhuanlan.zhihu.com/p/662015724>

经典面试题（更多面试题）

- 1) 什么是多模态模型？它和单模态模型有何区别？
- 2) AIGC（人工智能生成内容）的概念是什么？它在实际应用中有什么重要作用？
- 3) 多模态模型如何处理不同模态的数据（如文本、图像、语音等）？
- 4) 请简述Stable Diffusion的原理，并说明其相较于传统的生成对抗网络（GAN）有哪些优势。

学习资源

校招&社招

后端转 AI 应用求职训练营

系统学习 + AI 项目实战 + 求职包装

你可以获得：

- ✓ 定制后端转 AI 学习路线，不走弯路
- ✓ 企业级 AI 应用项目实战，直接写进简历
- ✓ AI 应用开发面试宝典，覆盖高频考点
- ✓ 1v1 简历包装，现有项目→AI应用项目
- ✓ 1v1 模拟面试 + AI 岗投递辅导

添加导师微信，
免费评估你的 AI 转型可行性



转 AI 应用开发，最快 2 周，平均一个月

视频教程

1) 李沐讲AI (动手学深度学习在线课程)

课程: <https://courses.d2l.ai/zh-v2/>

视频: <https://space.bilibili.com/1567748478?>

2) 马士兵-AI大模型全套教程

<https://www.bilibili.com/video/BV128cUe6EU2/>

3) AI大模型零基础全套教程

<https://www.bilibili.com/video/BV1uNk1YxEJQ>

4) 【清华NLP】刘知远团队大模型公开课

<https://www.bilibili.com/video/BV1UG411p7zv>

5) 斯坦福NLP cs224n

<https://web.stanford.edu/class/cs224n/>

6) 动手学深度学习

<https://zh.d2l.ai/>

7) 神经网络和深度学习

<https://nndl.github.io/>

8) 台大李宏毅-机器学习

<https://speech.ee.ntu.edu.tw/~hylee/ml/2023-spring.php>

9) 吴恩达 x Open AI ChatGPT 提示工程教程

https://www.bilibili.com/video/BV1s24y1F7eq/?vd_source=1c91f2eccd0fec95bb790e7f6942e73c

项目

1) 手把手教学从头build LLM

<https://github.com/rasbt/LLMs-from-scratch>

2) LlamaFactory: 一键式LoRA微调、全参SFT、增量预训练框架

<https://github.com/hiyouga/LLaMA-Factory>

3) 一个通用的pytorch模型训练模版工具

<https://github.com/lyhue1991/torchkeras/>

4) 学习LLM的非常好的项目

<https://github.com/karpathy/>

5) DeepSeek-R1 的复刻

<https://github.com/huggingface/open-r1/>

6) 蒸馏的 QwQ 的数据实现的 o1-like 模型。

<https://github.com/NovaSky-AI/SkyThought>

7) R1 多模态的复刻项目

<https://github.com/EvolvingLLMs-Lab/open-r1-multimodal>

书籍&课程

1) 面向开发者的 LLM 入门教程，吴恩达大模型系列课程中文版

<https://github.com/datawhalechina/llm-cookbook>

2) 大规模语言模型：从理论到实践

<https://intro-llm.github.io/>

3) Huggingface Transformers官方课程

<https://huggingface.co/learn/nlp-course/chapter1/1>

4) Transformers快速入门（快速调包BERT系列）

<https://transformers.run/>

5) 神经网络和深度学习

<https://nndl.github.io/>

6) LLM 资料库

<https://github.com/morsoli/llm-books>

7) 大模型基础

<https://github.com/datawhalechina/so-large-lm>

8) 基于《斯坦福大学大规模语言模型课程》打造的大模型理论教程

<https://datawhalechina.github.io/so-large-lm>

9) 动手学大模型应用开发

<https://github.com/datawhalechina/llm-universe>

10) 基于MetaGPT的多智能体入门与开发教程

<https://github.com/datawhalechina/hugging-multi-agent>

11) Transformers 教程

<https://transformers.run/>

技术分析文章

1) 图解Transformer

<https://jalammar.github.io/illustrated-transformer/>

2) GPT2图解

<https://jalammar.github.io/illustrated-gpt2/>

3) GPT3分析

<https://jalammar.github.io/how-gpt3-works-visualizations-animations/>

4) 如何合成微调数据

<https://eugeneyan.com/writing/synthetic/>

5) 大模型量化解析

<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>

6) OpenAI 的思考模型最佳实践

<https://platform.openai.com/docs/guides/reasoning-best-practices>

知识库

1) 鱼皮的 AI 知识库

<https://ai.codefather.cn/>

2) GeekAGI知识库

<https://geek-agi.feishu.cn/wiki/B9rYwwg6xidZYJkbrlscxTQFnOc>

3) Open 1 +X AI通识课

<https://datawhaler.feishu.cn/wiki/X9AVwtmvyi87blkYpi2cNGLln3v>

4) 一站式AI产品经理入门指南

<https://v11enp9ok1h.feishu.cn/wiki/KilvwdFOciiqqNkwKzTcmn88ndL>

论文报告

1) DeepSeek R1

https://github.com/deepseek-ai/DeepSeek-R1/blob/main/DeepSeek_R1.pdf

2) Llama技术报告

<https://arxiv.org/abs/2407.21783>

3) Mixtral-8X7B-MOE

<https://mistral.ai/news/mixtral-of-experts>

4) 70B模型训练细节

<https://imbue.com/research/70b-intro/>

5) Qwen技术报告

<https://arxiv.org/abs/2309.16609>

6) 面壁智能的详尽技术报告

<https://arxiv.org/abs/2404.06395>

7) VidProM: 文生视频提示数据集

<https://arxiv.org/abs/2403.06098>

8) 语言模型的算法进展

<https://arxiv.org/abs/2403.05812>

9) 大模型推理速度计算和瓶颈分析

<https://arxiv.org/abs/2405.08944>

10) Prompt 工程综述

<https://arxiv.org/abs/2407.12994>

11) 高级 RAG 优化方法

<https://arxiv.org/abs/2407.21059>

12) Kimi K1.5 推理模型

<https://arxiv.org/abs/2501.12599v1>

13) DeepSeek Math

<https://arxiv.org/abs/2402.03300>

14) 中文蒸馏数据集

<https://huggingface.co/datasets/Congliu/Chinese-DeepSeek-R1-Distill-data-110k>

15) Qwen2 技术报告

<https://arxiv.org/abs/2407.10671>

16) LLM 评估, 不应忽视非确定性

<https://arxiv.org/abs/2407.10457>

程序员必备资源

- 1) 程序员学习交流圈: [极客教程、实战项目、求职宝典](#)
- 2) 程序员面试八股文: [实习/校招/社招高频考点、企业真题解析](#)
- 3) 程序员写简历神器: [专业模板、丰富例句、直通面试](#)
- 4) AI 知识资源大全: [前沿技术、最新 AI 资讯、提示词大全](#)
- 5) 1 对 1 模拟面试: [实习/校招/社招面试拿 Offer 必备](#)